Does a planning tool help LLMs on PDDL tasks? — unified findings

5 language models · 5 PDDL tasks · no-tools vs +tool vs +tool+nudge · think=off headline + think=off × on cross-mode synthesis · Wilson 95% CIs · success rates end-to-end

The headline — two of five tasks do not happen without the tool

Tracking world state through a plan (simulate) and solving (solve) essentially require the tool:

0%

simulate, no-tools — every model, BOTH think modes, all 3,000 trials

83–97%

simulate, tool + one steering sentence (≥9B, both modes)

38% vs 92%

solve — BEST unaided configuration (Qwen3.6-35B, think=on) vs the same cell with the tool

The solve baseline is quoted at its best on purpose — reasoning mode lifts the unaided 35B from 9% to 38%, and the tool still adds +53pp in that same mode; across both modes the benefit floors at ≥ +46pp for every ≥9B model. Where the unaided model is already strong, the honest mode-invariant benefit is modest: +5…+16pp (validate_plan), +20…+25pp (validate_problem), ≥ +21pp (validate_domain). The deck quantifies the pattern, its mechanism — tool-CALLING, not tool output, is what fails — and the token bill.

Scorecard — six research questions, six verdicts

RQ	question	verdict	evidence (≥9B, think=off)
RQ0.1	does the tool help validate domain/problem files?	YES	availability +21…+30pp; 3/3 sig-for, 0/3 sig-against
RQ0.2	does the tool help SOLVE?	YES	availability +54…+92pp; 3/3 sig-for, 0/3 sig-against
RQ0.3	does the tool help check a plan?	MIXED	availability -67…+13pp; 1/3 sig-for, 2/3 sig-against
RQ0.4	does the tool help track state (simulate)?	YES	availability +65…+92pp; 3/3 sig-for, 0/3 sig-against
RQ0.5	does the advantage grow with plan length?	YES	validate_plan gap widens +5 → +7 → +27pp
RQ0.6	does the advantage track object count?	NO	validate_problem gap flat (+22 / +23 / +23pp)

Verdicts locked + asserted against the computed signed CI counts (think=off, the clean read). The SAME signed rule on the think=on corpus reproduces the pattern — YES / YES / MIXED / YES, RQ0.5 YES, RQ0.6 NO (asserted at build time); the MAGNITUDES move with the decode budget (cross-mode section).

What we're testing — five PDDL jobs, three setups

PDDL is the standard formal language AI planners use to describe a world, its actions, a goal, and step-by-step plans. We give a language model five PDDL jobs:

- solve — find a plan reaching the goal · simulate — track world state as a plan runs, action by action
- validate_domain / validate_problem — check the PDDL world/problem files are correct · validate_plan — yes/no: does a given plan actually work?

Every job runs in three setups ("arms"), never mixed:

- no-tools — the model answers alone (the baseline) · tool available — a real planner/validator it MAY call · tool + nudge — plus one sentence telling it to use the tool ("steered").
- no-tools vs tool-available asks: does merely HAVING the tool help? tool-available vs steered asks: did the model just need to be TOLD?
- Short codes in tables: nt-neut · tl-neut · tl-ster (same order).

How to read the results

- success rate — how often the model's answer matched the known-correct answer, 0–100%. It is the only score shown; arms are scored separately and never pooled.
- Error bars (charts) and [low, high] brackets (tables) are Wilson 95% confidence intervals. When two ranges don't overlap, the difference is real rather than noise; a “*” on a gap marks exactly that. Green = the tool helped, red = the tool hurt.
- “≥9B” — headline conclusions use the larger models (Qwen3.5-9B, Gemma-MoE-26B, Qwen3.6-35B); Qwen3.5-4B is shown for context and the 0.8B failure mode gets its own slide.
- Headline numbers are think=off (the model answers directly) — the clean, unconfounded read. The cross-mode section then asks what survives think=on (the model reasons first): reasoning + answer SHARE one 8,192-token decode budget there, so raw think=on gaps are budget-confounded and cross-mode claims are made only through budget-insensitive statistics (the robust floor).
- “difficulty bins” (RQ0.5/0.6) — problems split into easy / medium / hard thirds, no-tools vs tool + nudge, to see whether the tool's advantage changes as problems get harder.
- A contamination re-run on disguised problems (sweep-6) left the no-tools baseline unchanged (dedicated slide near the end).

Why the verdicts count direction — signed significance

A gap counts toward a YES only when the two arms' 95% intervals are disjoint AND the gap points the hypothesised way (the tool helps). Significant gaps pointing the other way are tallied separately as significant-AGAINST — they are findings, not noise.

- The case that makes this matter: on validate_plan, merely making the tool available moves Gemma-MoE-26B by -67pp — significant, and AGAINST the tool (it stops answering).
- A sign-blind test would count that gap as evidence the tool 'has an effect' and read RQ0.3 as YES. The signed rule reads it as MIXED — which is what the data actually says.
- The build asserts the computed signed verdicts equal the locked scorecard (think=off) AND that the same rule reproduces the verdict pattern on the think=on corpus — the deck cannot silently drift from this rule in either mode.

RQ0.1 — validate_domain + validate_problem

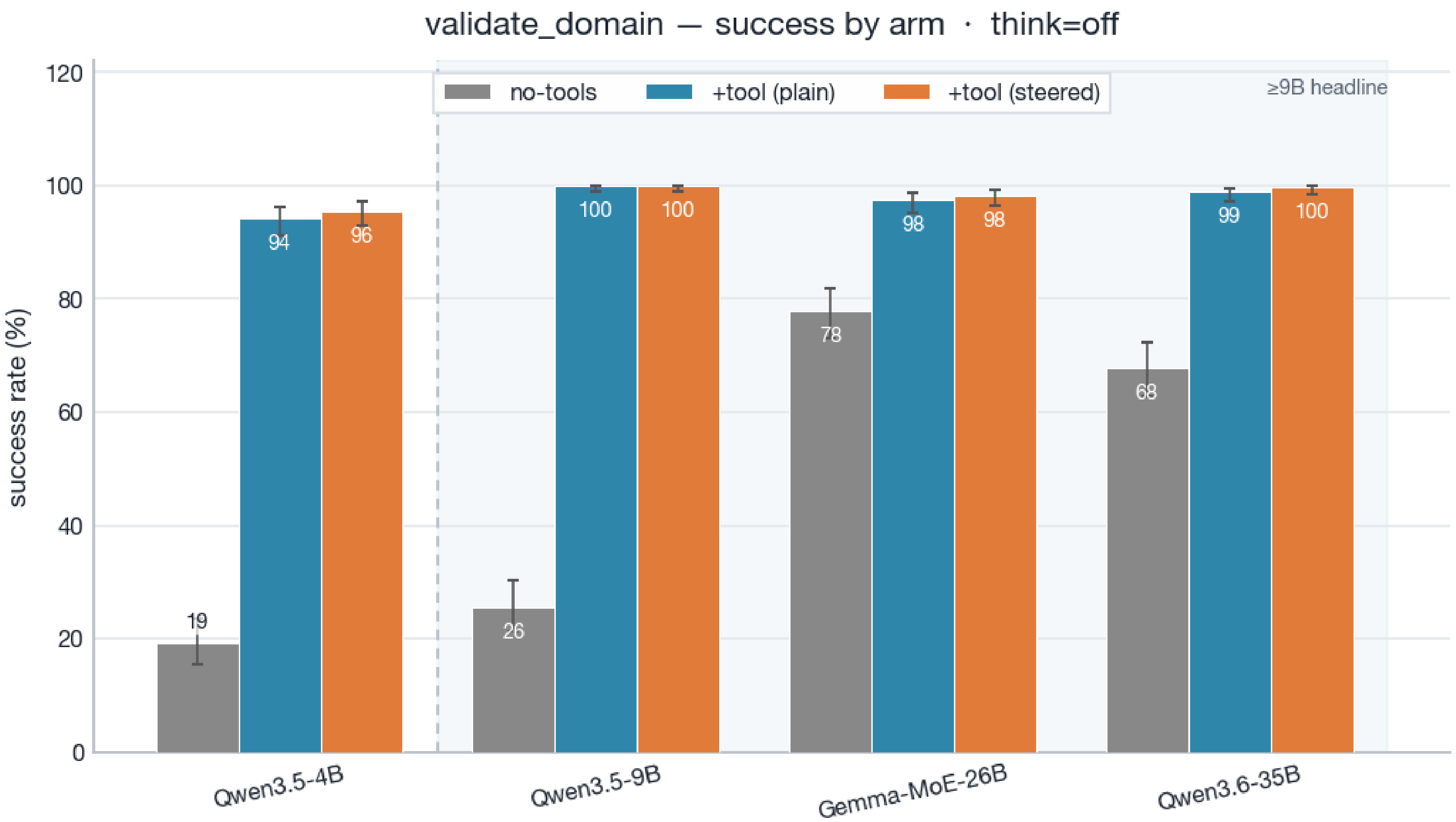
YES

Does giving the model a validation tool help it validate PDDL domains and problems? (validate_domain + validate_problem)

- Availability (validate_problem, ≥9B): gap +21…+30pp; 3/3 favorable-significant, 0/3 significant-against (95% CIs).
- Steering (validate_problem, ≥9B): gap -5…+1pp; 0/3 favorable-significant.

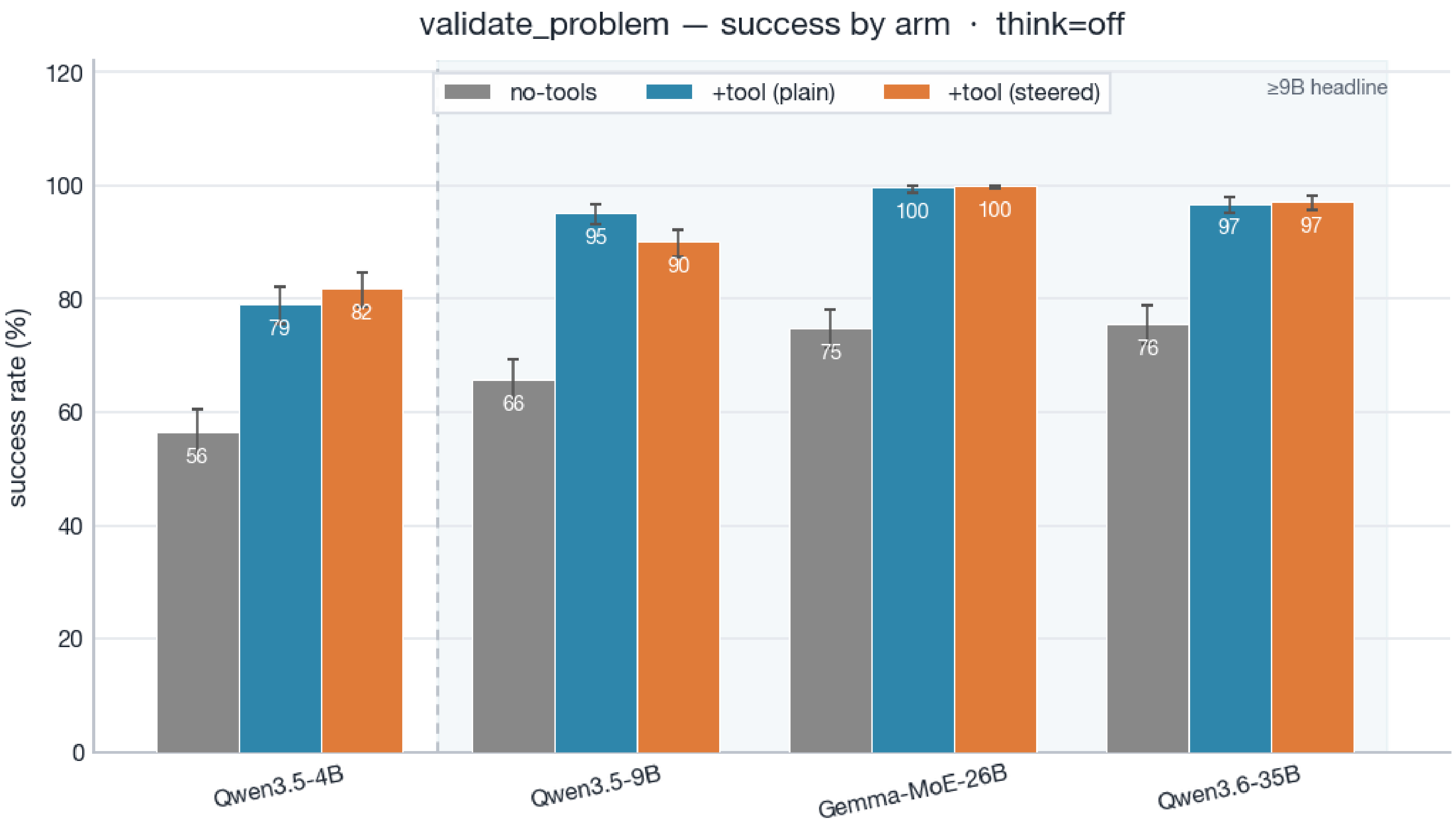
Caveat: at 0.8B the availability gap REVERSES on validate_problem (−25pp) — the smallest model mishandles the tool (see the small-model slide). YES holds from 4B up.

RQ0.1 — Evidence: validate_domain success by arm



Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide). Full table in the backup.

RQ0.1 — Evidence: validate_problem success by arm



Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide). Full table in the backup.

RQ0.2 — solve

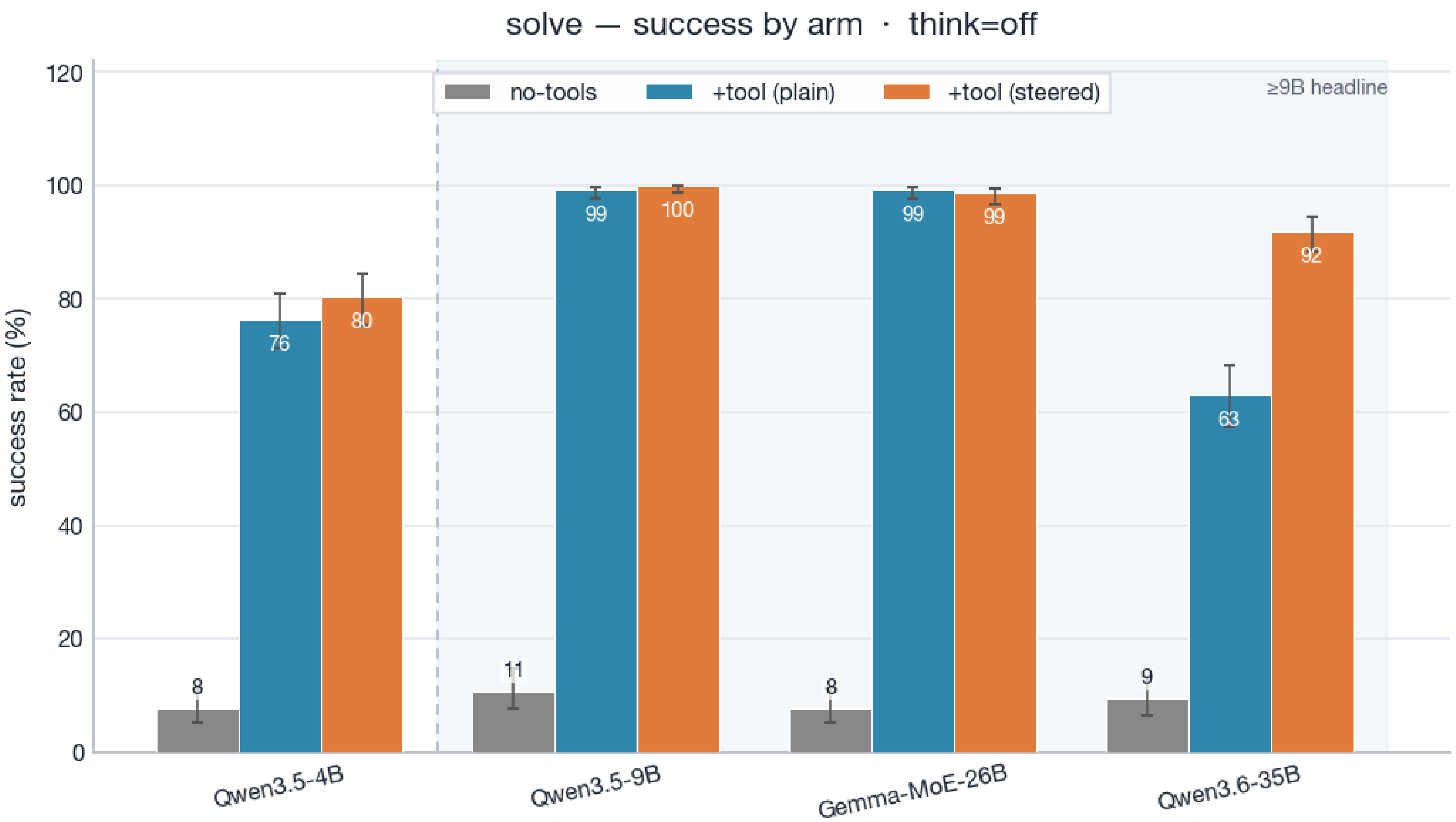
YES

Does giving the model a planning tool help it SOLVE PDDL problems? (solve)

- Availability (solve, ≥9B): gap +54…+92pp; 3/3 favorable-significant, 0/3 significant-against (95% CIs).
- Steering (solve, ≥9B): gap -1…+29pp; 1/3 favorable-significant.

Decisive: no-tools is floored (~8–11%); +tool lifts ≥9B to 63–99%. Steering adds a large further lift where plain left headroom (Qwen3.6-35B +29pp).

RQ0.2 — Evidence: solve success by arm

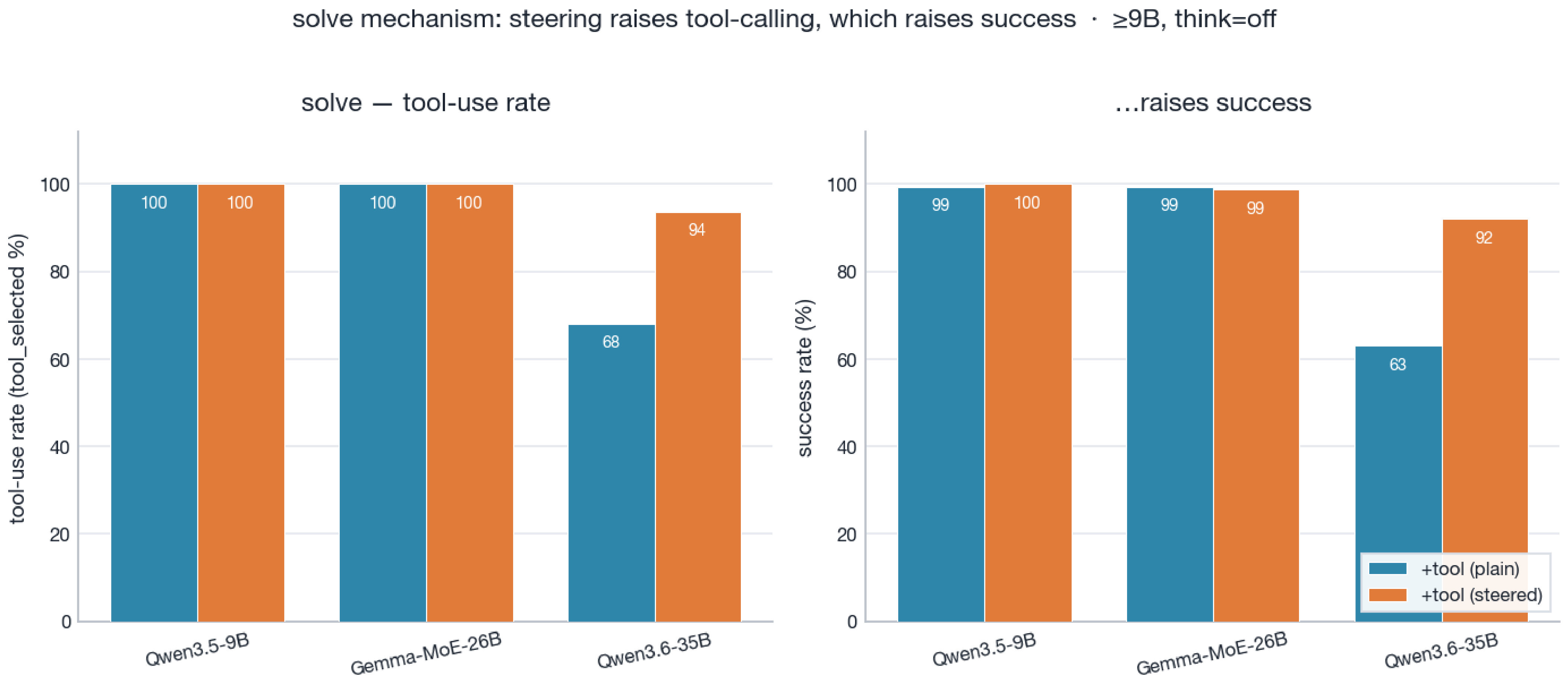


Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.2 — solve success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	8 [5,11]	76 [71,81]	80 [75,84]	**+69***	+4
Qwen3.5-9B	11 [8,15]	99 [98,100]	100 [99,100]	**+89***	+1
Gemma-MoE-26B	8 [5,11]	99 [98,100]	99 [97,99]	**+92***	-1
Qwen3.6-35B	9 [7,13]	63 [57,68]	92 [88,95]	**+54***	**+29***

RQ0.2 — Mechanism: steering raises tool-calling → raises success



solve, ≥9B. Left: tool-use rate (tool_selected) plain vs steered. Right: success. Where +tool(plain) under-calls the tool, steering raises tool-calling, and success rises with it — the tool's value is gated on the model actually invoking it.

RQ0.3 — validate_plan

MIXED

Does giving the model a validation tool help it CHECK whether a plan is valid? (validate_plan)

- Availability (validate_plan, ≥9B): gap -67…+13pp; 1/3 favorable-significant, 2/3 significant-against (95% CIs).
- Steering (validate_plan, ≥9B): gap +3…+72pp; 3/3 favorable-significant.

MIXED: the model alone is already strong on validate_plan (75–90%). At +tool(plain) the availability gap is significant-AGAINST for Gemma-MoE (−67pp: it stops answering) and Qwen3.6-35B (−9pp); only Qwen3.5-9B is favorable. Steering RECOVERS and beats no-tools (Gemma 21→93%), but the net tool benefit over a strong baseline is small/mixed.

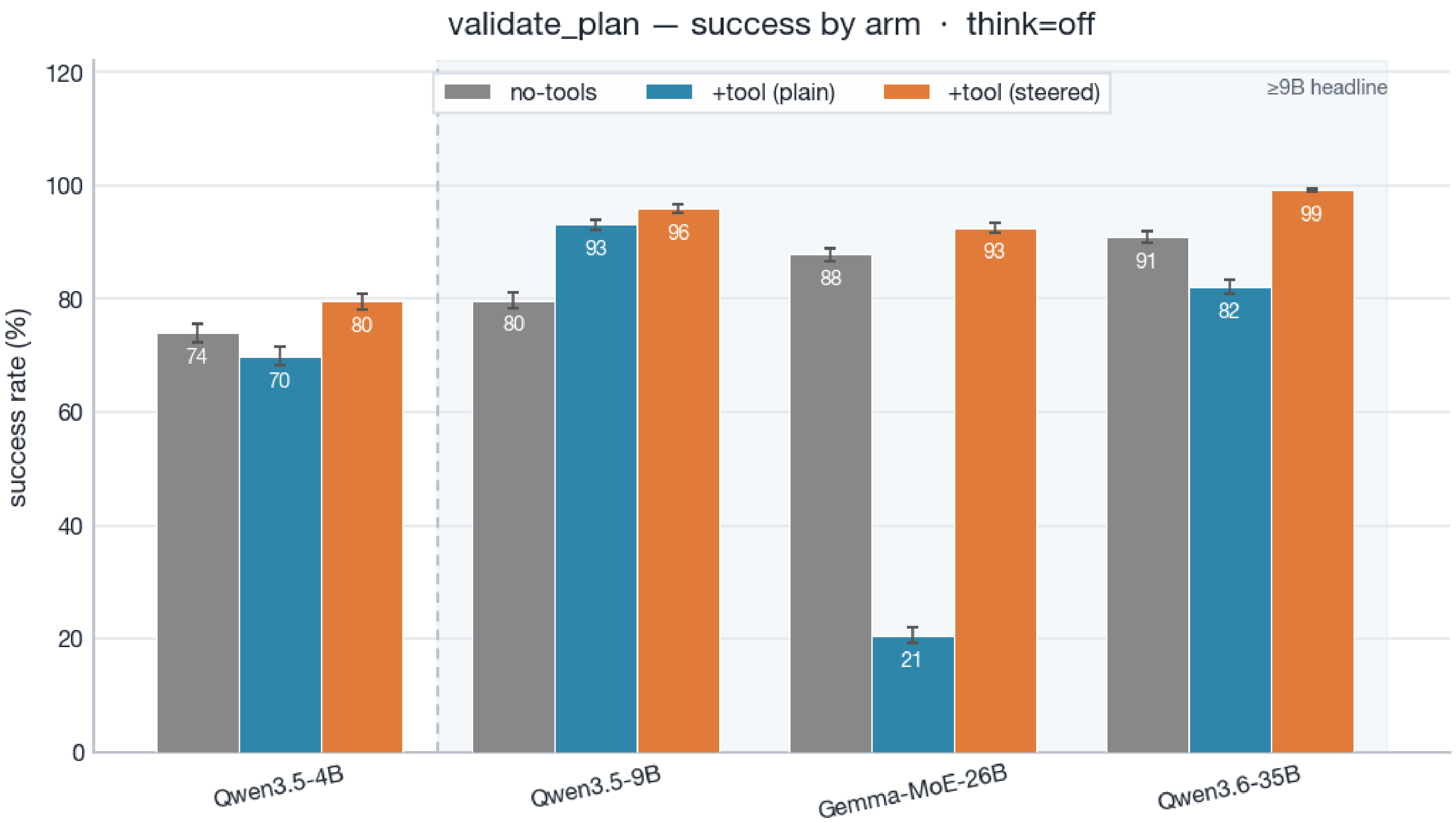
Reading guide — the low +tool(plain) bar is silence, not error

On validate_plan the +tool(plain) arm often produces NO verdict at all — the model fails to call the tool and answers nothing. When it does answer, it is near-perfect:

- Gemma-MoE-26B — raw success 21%, but it only produces a verdict on 21% of trials; when it DOES answer, accuracy is 99% (false-positives: 5).
- Qwen3.5-0.8B — raw success 23%, but it only produces a verdict on 34% of trials; when it DOES answer, accuracy is 68% (false-positives: 2).

→ So the collapse is a tool-CALLING failure, not a verdict failure — and steering, which raises tool-calling, repairs it (mechanism slide after the evidence).

RQ0.3 — Evidence: validate_plan success by arm

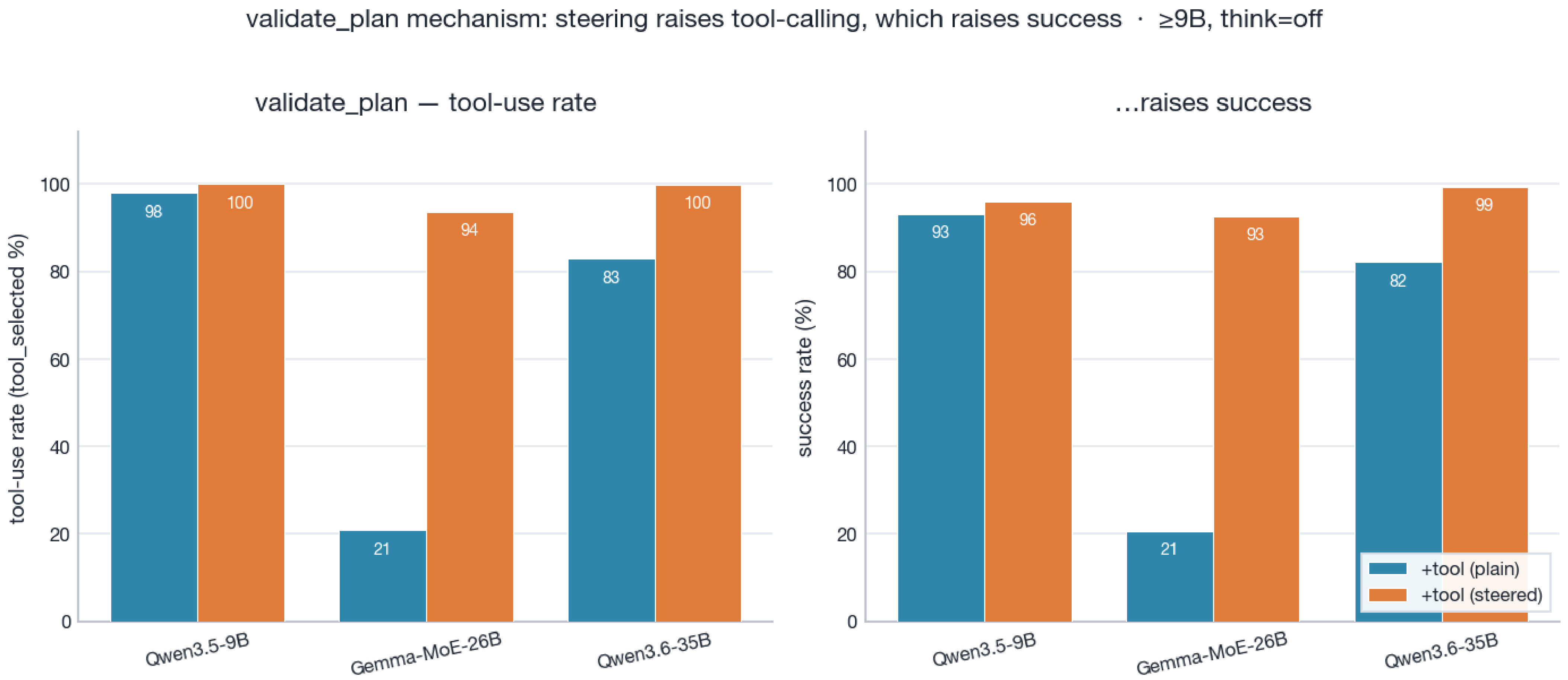


Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.3 — validate_plan success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	74 [72,76]	70 [68,72]	80 [78,81]	-4*	+10*
Qwen3.5-9B	80 [78,81]	93 [92,94]	96 [95,97]	+13*	+3*
Gemma-MoE-26B	88 [87,89]	21 [19,22]	93 [92,93]	-67*	+72*
Qwen3.6-35B	91 [90,92]	82 [81,83]	99 [99,100]	-9*	+17*

RQ0.3 — Mechanism: steering raises tool-calling → raises success



validate_plan, ≥9B. Left: tool-use rate (tool_selected) plain vs steered. Right: success. Where +tool(plain) under-calls the tool, steering raises tool-calling, and success rises with it — the tool's value is gated on the model actually invoking it.

RQ0.4 — simulate

YES

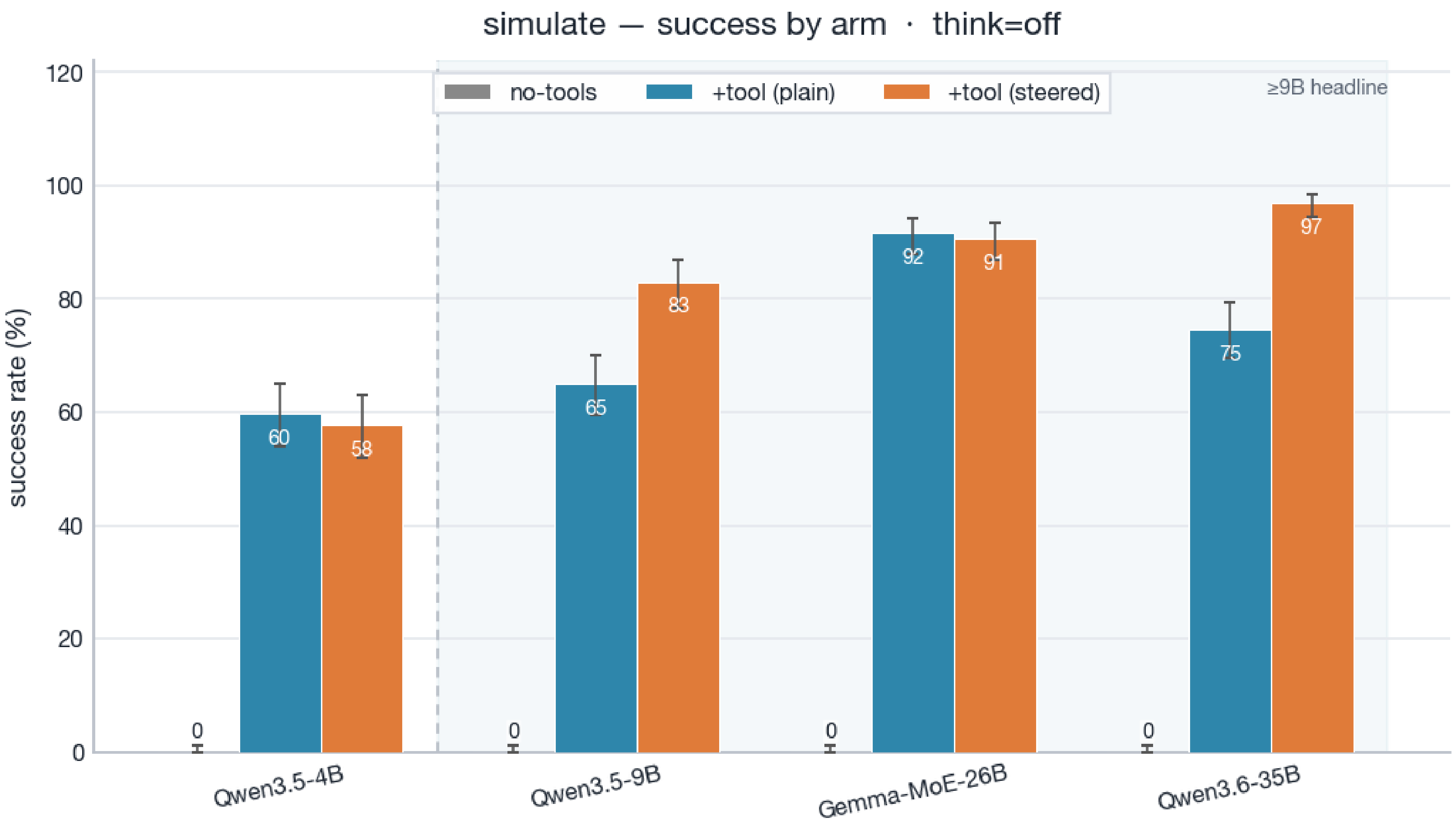
Does giving the model a simulation tool help it TRACK state (simulate a plan)? (simulate)

- Availability (simulate, ≥9B): gap +65…+92pp; 3/3 favorable-significant, 0/3 significant-against (95% CIs).
- Steering (simulate, ≥9B): gap -1…+22pp; 2/3 favorable-significant.

Decisive: no-tools is 0% everywhere (state-tracking by hand fails); +tool reaches 65–92% on ≥9B, with steering adding +18–22pp.

Grader (strict, end-to-end, no partial credit): success = canonical-form deep-equality of the FULL state trajectory against the oracle — structured JSON only, no free-text fallback; normalisation removes formatting variance, not semantic error. The unaided 0% decomposes (≥9B, all failures): 68% unparseable trajectory JSON, 29% truncated at the 8,192 cap, 3% parsed but wrong trajectory.

RQ0.4 — Evidence: simulate success by arm



Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide). Full table in the backup.

Why does 9B beat 35B? — tool-call propensity, not capability

task	model	mode	tool-use % (plain)	success % (plain)	success % (steered)
solve	Qwen3.5-9B	think=off	100	99	100
	Qwen3.5-9B	think=on	59	58	73
	Qwen3.6-35B	think=off	68	63	92
	Qwen3.6-35B	think=on	78	78	92
validate_plan	Qwen3.5-9B	think=off	98	93	96
	Qwen3.5-9B	think=on	69	65	93
	Qwen3.6-35B	think=off	83	82	99
	Qwen3.6-35B	think=on	100	99	99

The inversion exists only in the PLAIN arm (think=off solve: 9B 99% vs 35B 63%) and success tracks tool-use almost 1:1 — the dominant failure is tool_not_selected, and accuracy-when-calling is ≥93% for BOTH models. One steering sentence closes it (35B solve 63→92%). The propensity FLIPS with mode (think=on: 35B calls validate_plan 99.9%, 9B drops to 69%) — model- and mode-idiosyncratic behaviour, not a size law; unaided baselines favour 35B on every validation task. No prior work pins this size inversion; closest are the tool over-reliance/over-refusal duality reports.

Small-model caveat — Qwen3.5-0.8B mishandles the tool

task	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
solve	0 [0,1]	12 [9,16]	14 [11,18]	**+12***	+2
validate_domain	81 [76,85]	88 [84,91]	89 [86,92]	+7	+2
validate_problem	51 [47,55]	26 [23,30]	18 [16,22]	**-25***	**-8***
validate_plan	50 [48,52]	23 [22,25]	25 [23,26]	**-27***	+2
simulate	0 [0,1]	6 [4,10]	3 [1,5]	**+6***	-4

The only model where tool AVAILABILITY reverses sign (validate_problem −25pp*, validate_plan −27pp*). Mechanism (trial-level failure_reason, both reversal tasks × both tool arms, think=off): it SELECTS the tool in 93–100% of trials but cannot drive it — 57% of all trials end in tool_error (98% of those are missing_required_arg: the right tool called with a required argument omitted) and a further 6% exhaust the turn loop without an answer. Externally consistent: MCPToolBench++ reports the same small-model shape — plausible tool calls (AST 0.6–0.9) that fail to execute (Pass@1 0.2–0.5).

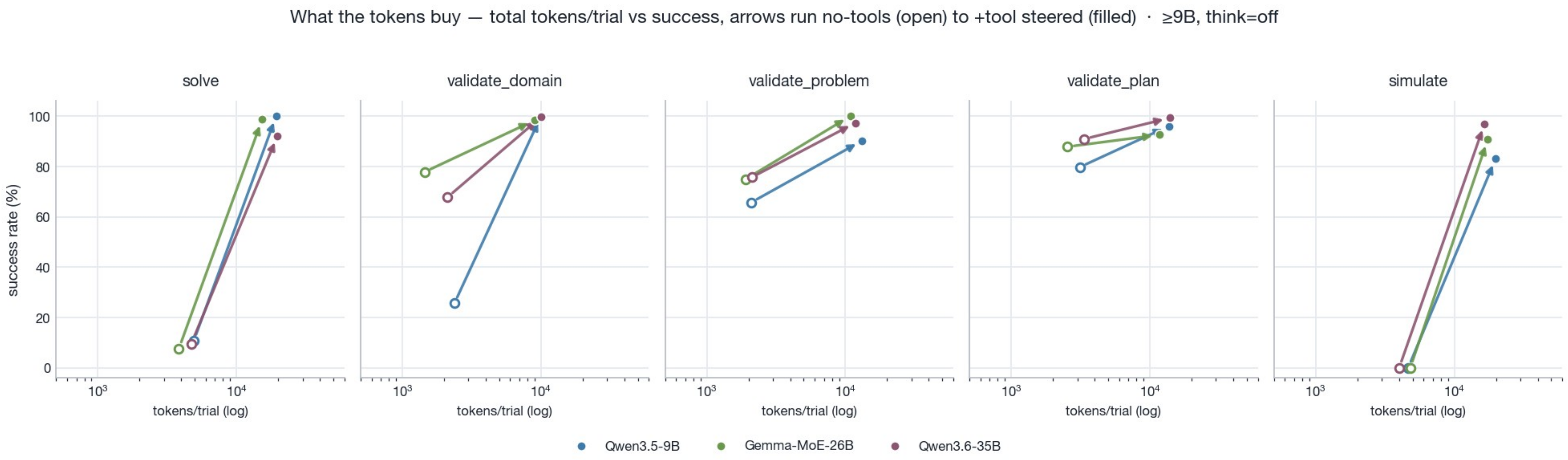
What do tools cost? — token consumption

Token cost = TOTAL tokens a trial consumes — input (prompt) + output (completion), summed across the model's turns. The whole bill, not just generated text. Scope: ≥9B, think=off.

- Tools raise per-trial consumption ~4–15× — but consumption alone is half the story: the next figure holds cost and success in one view so neither number is quoted without the other.
- Quality-adjusted efficiency is cost-of-pass = total tokens ÷ correct answers: every token burned on a failed attempt is charged to the successes. LOWER is better.

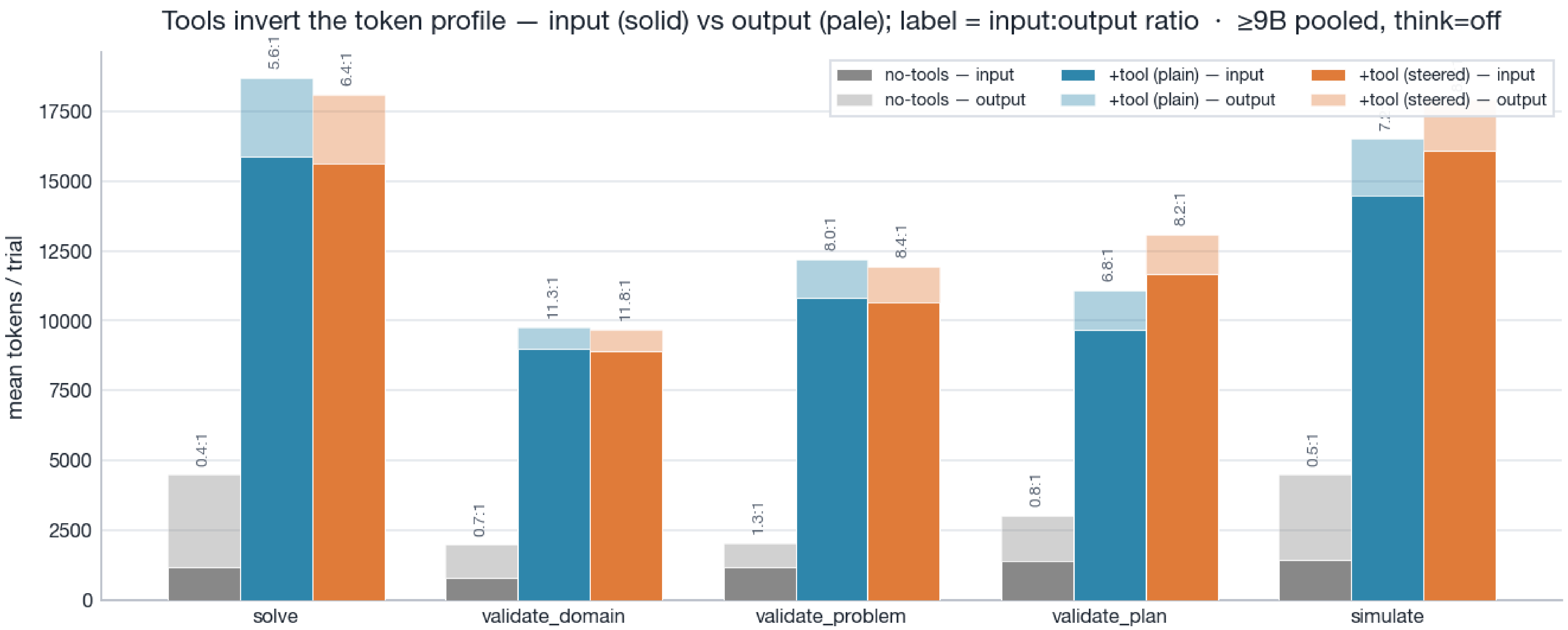
→ Caveats: (1) prefix caching (~90% on this roster) makes the re-sent tool INPUT cheap in server compute — but it is still real context-budget consumption, and tool OUTPUTS across turns are novel/uncached; the raw total is the honest accounting number. (2) Output is right-censored at the 8,192-token cap with arm-dependent truncation (evidence slide at the end of this section). (3) A completion-only view flatters tools — kept as a labelled secondary lens in the backup.

What the tokens buy — pay ~4–15× per trial, buy +5…+100pp



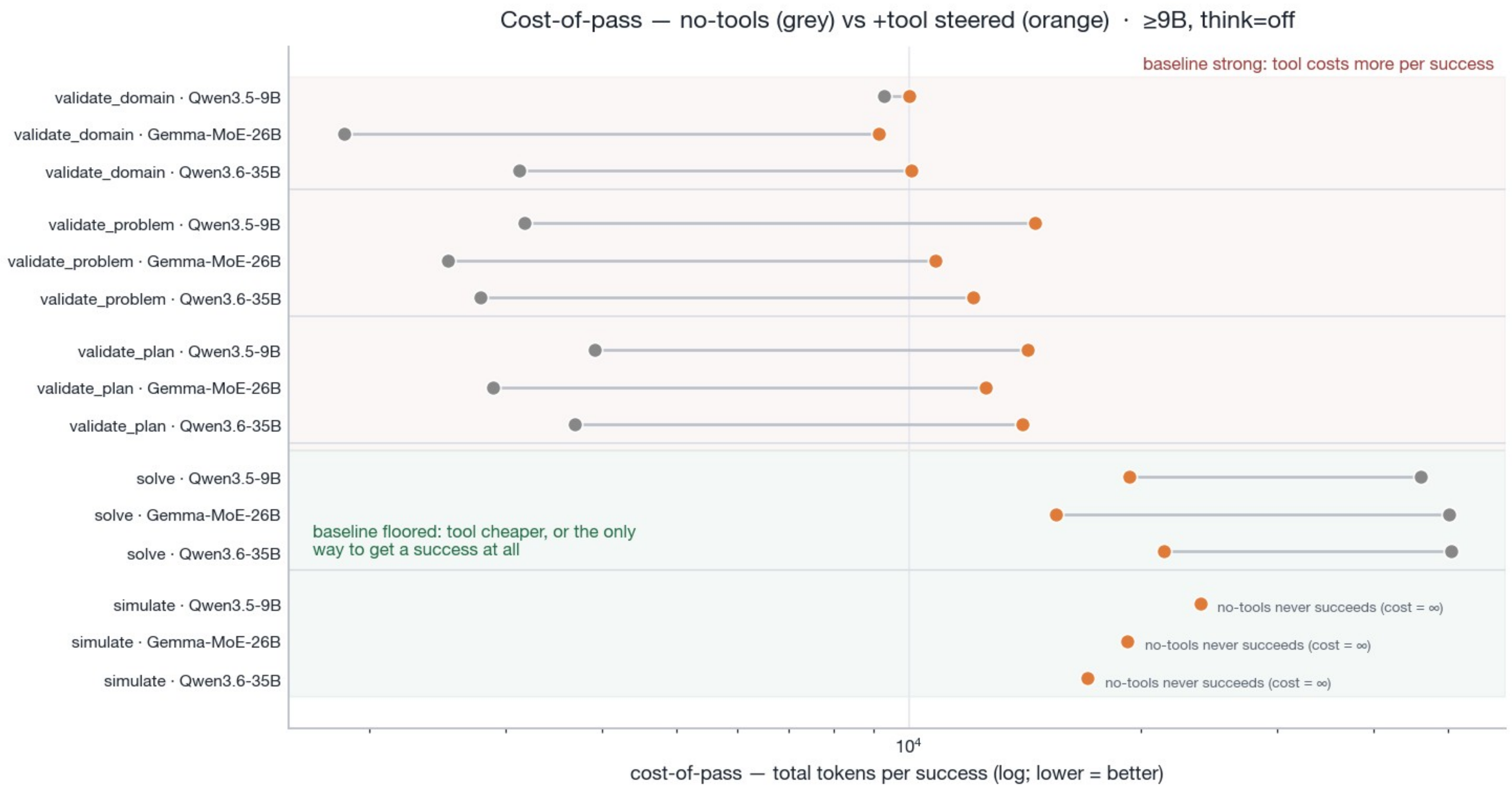
Each arrow is one ≥9B model on one task, from no-tools ○ to +tool(steered) ● — x = mean total tokens/trial (log), y = success. Every arrow points up-and-right: the tool always costs more per trial; the success it buys is decisive where the baseline is floored (solve, simulate) and modest where the baseline is already strong (validate_*). Per-model tables in the backup.

Tools invert the token profile — the cost is input, not output



Mean input (solid) vs output (pale) tokens per trial, ≥9B pooled. no-tools is output-heavy (~0.3:1 input:output — it reasons in the open); the tool arms are input-heavy (~5:1) because tool schemas + tool outputs are re-fed every turn. That re-fed input is the real token cost of tools — an output-only comparison hides it entirely, which is why the headline metric is the total.

Cost-of-pass — the tool pays for itself exactly where the model can't do the task



Total tokens per success (log; lower = better), no-tools ○ vs +tool(steered) ●, bootstrap point estimates. Where the baseline is strong (validate_*) the tool is ~3–11× costlier per success; where the baseline is floored, the tool is ~3× cheaper (solve) or the only source of successes at all (simulate — no-tools never succeeds, so its cost-of-pass is infinite).

Why efficiency moves — (tokens/trial) ÷ (more often right) = cost-per-success

task	model	tokens/trial ×	more often right ×	= cost-per-success ×
solve	Qwen3.5-4B	4.4× costlier	10.5× more right	0.4× cheaper
	Qwen3.5-9B	3.9× costlier	9.4× more right	0.4× cheaper
	Gemma-MoE-26B	4.0× costlier	12.9× more right	0.3× cheaper
	Qwen3.6-35B	4.2× costlier	9.9× more right	0.4× cheaper
validate_domain	Qwen3.5-4B	11.0× costlier	5.0× more right	2.2× costlier
	Qwen3.5-9B	4.2× costlier	3.9× more right	1.1× costlier
	Gemma-MoE-26B	6.2× costlier	1.3× more right	4.9× costlier
	Qwen3.6-35B	4.7× costlier	1.5× more right	3.2× costlier
validate_problem	Qwen3.5-4B	14.9× costlier	1.4× more right	10.3× costlier
	Qwen3.5-9B	6.3× costlier	1.4× more right	4.6× costlier
	Gemma-MoE-26B	5.7× costlier	1.3× more right	4.3× costlier
	Qwen3.6-35B	5.6× costlier	1.3× more right	4.4× costlier
validate_plan	Qwen3.5-4B	8.7× costlier	1.1× more right	8.1× costlier
	Qwen3.5-9B	4.4× costlier	1.2× more right	3.6× costlier
	Gemma-MoE-26B	4.6× costlier	1.1× more right	4.3× costlier
	Qwen3.6-35B	4.2× costlier	1.1× more right	3.8× costlier

Are the token numbers comparable? — truncation, turns, latency proxy

task	arm	hit cap %	cap-hit & failed %	mean turns	output tok/trial	total tok/trial
solve	no-tools	16%	15%	1.0	3.3k	4.5k
	+tool (plain)	40%	6%	2.7	2.8k	19k
	+tool (steered)	34%	3%	2.7	2.5k	18k
validate_domain	no-tools	4%	4%	1.0	1.2k	2.0k
	+tool (plain)	0%	0%	2.0	792	9.8k
	+tool (steered)	0%	0%	2.0	757	9.7k
validate_problem	no-tools	2%	2%	1.0	871	2.0k
	+tool (plain)	1%	0%	2.1	1.4k	12k
	+tool (steered)	0%	0%	2.1	1.3k	12k
validate_plan	no-tools	1%	1%	1.0	1.6k	3.0k
	+tool (plain)	7%	1%	1.7	1.4k	11k
	+tool (steered)	8%	1%	2.0	1.4k	13k
simulate	no-tools	29%	29%	1.0	3.0k	4.5k
	+tool (plain)	67%	10%	2.0	2.0k	17k
	+tool (steered)	73%	9%	2.2	1.8k	18k

'hit cap %' (any turn) is not comparable across arms: no-tools is single-turn (cap ⇒ no answer); tool arms are multi-turn and graded from the tool result, so most cap-hit tool trials still succeed — 'cap-hit & failed %' is the truncation that mattered.

RQ0.5 — does the advantage track difficulty?

YES

Does the planning tool's advantage CHANGE as instances get harder (longer reference / plan length)?

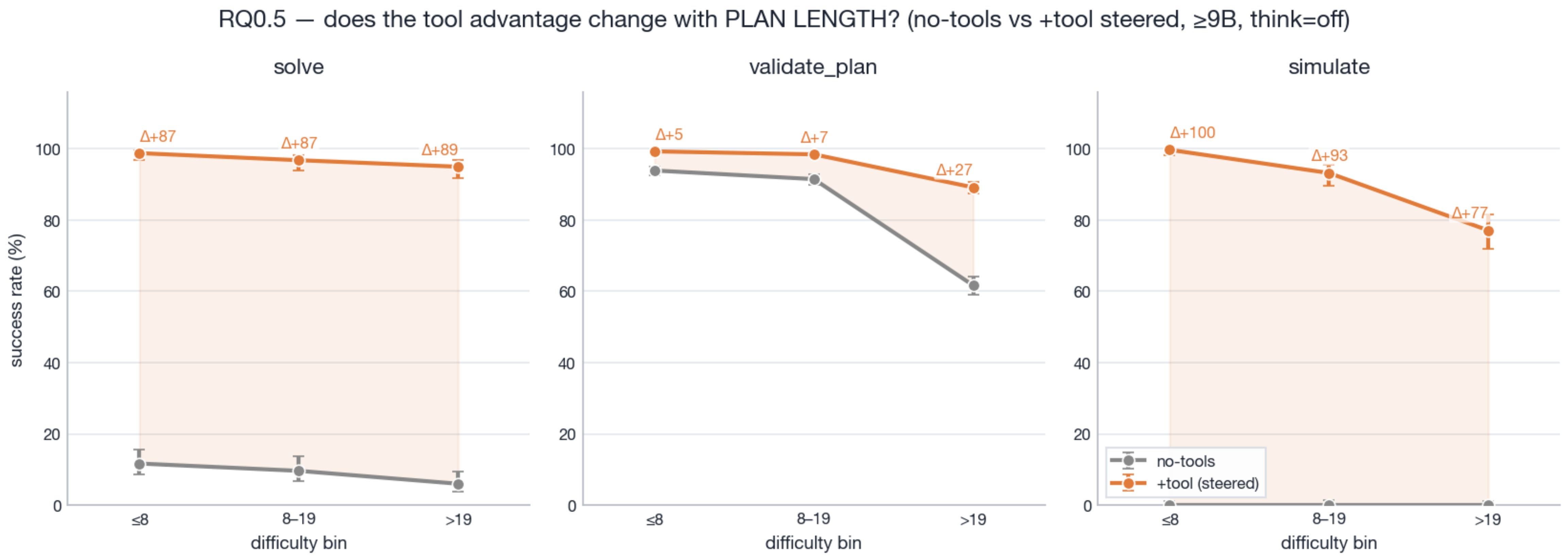
YES for validate_plan — the advantage GROWS with plan length.

Headroom case (validate_plan) gap by bin: ≤8: Δ+5 → 8–19: Δ+7 → >19: Δ+27.

- validate_plan is the headroom-gated case (both arms have room to move): the no-tools − tool gap widens from +5pp (short plans) to +27pp (long plans) as no-tools degrades while the tool holds. This is the generalisation claim: the harder the instance, the more the tool is worth.
- solve and simulate bracket that case. solve's gap is large but roughly CONSTANT (+87→+87→+89pp) — both arms pinned (no-tools floored ~6–12%, tool near-ceiling ~95–99%). simulate's gap is large but DECLINES (+100→+93→+77pp, CI-disjoint): the no-tools arm stays floored at 0%, but the tool arm itself degrades on long trajectories (99.7→93.2→77.1%) — the one place tool-assisted success erodes with difficulty.
- Mode note: under think=on the headroom case MOVES to solve (+48→+65pp with length) — reasoning runs out of budget exactly where plans get long; reported as a finding in the cross-mode section, not a contradiction.

Phase-2 is headroom-gated: an advantage can only be seen to CHANGE where both arms have room to move. ≥9B, think=off, no-tools vs +tool(steered).

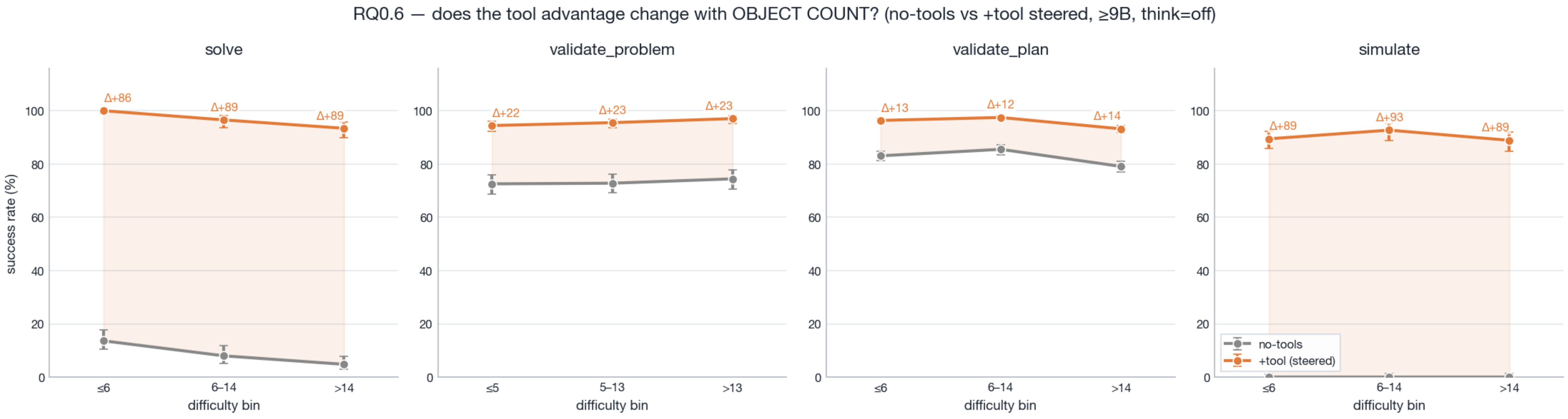
RQ0.5 — Evidence



Per difficulty bin: no-tools (grey) vs +tool steered (orange), Wilson 95% CIs; Δ = tool − no-tools over each bin (shaded = the advantage). Full bin table in the backup.

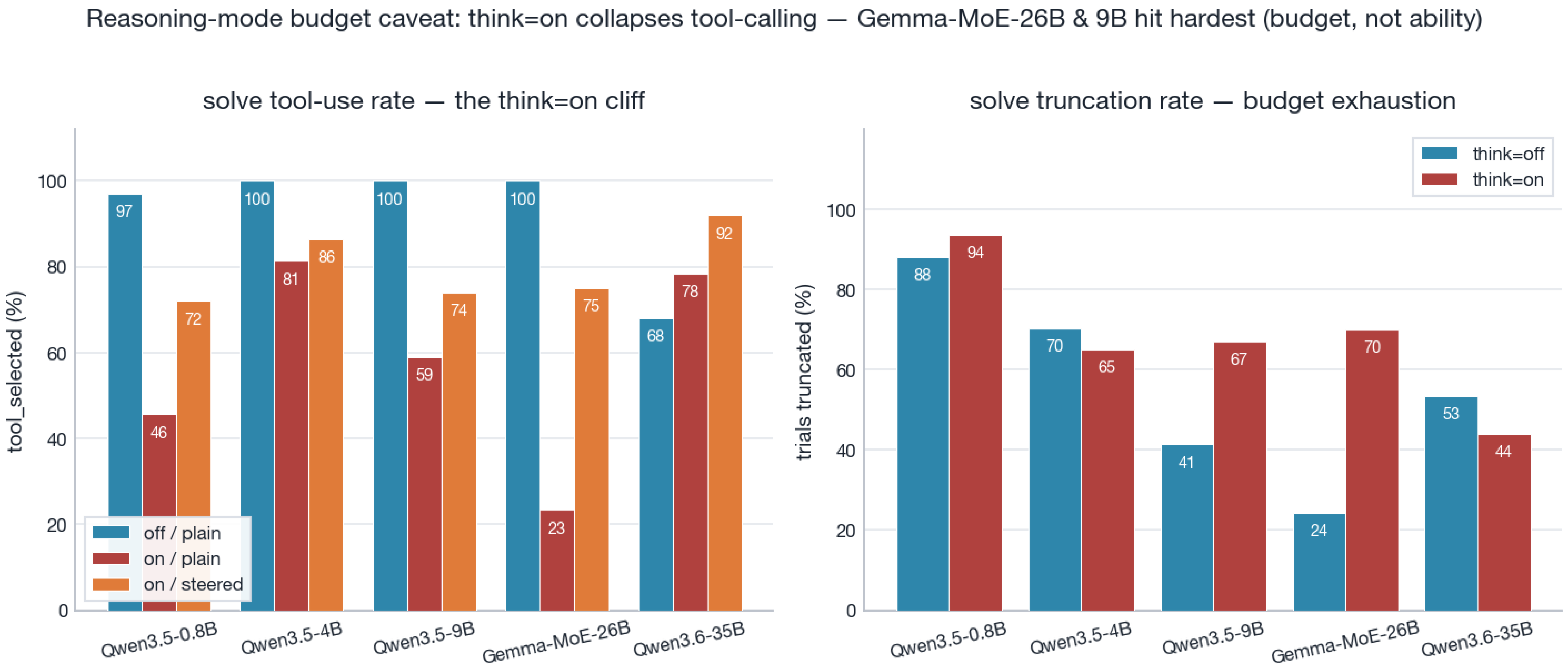
RQ0.6 — object count does not move the advantage

NO



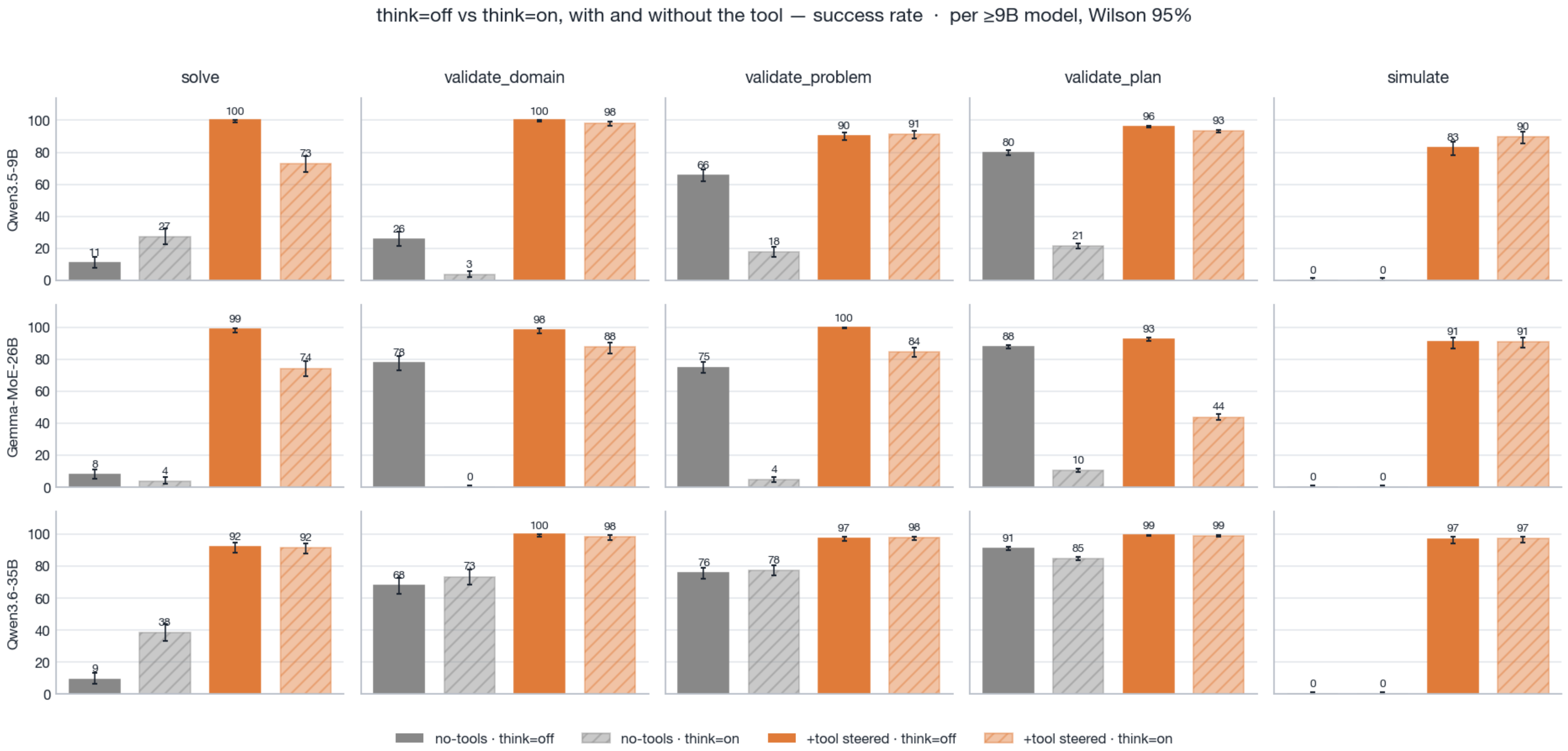
Headroom case (validate_problem): gap ≤5: Δ+22 → 5–13: Δ+23 → >13: Δ+23pp — essentially flat; the other tasks are also flat (no-tools floored or already separated). Object count is not a difficulty axis the tool's advantage tracks. Full bin table in the backup.

Does it survive reasoning mode? — read the decode-budget cliff first



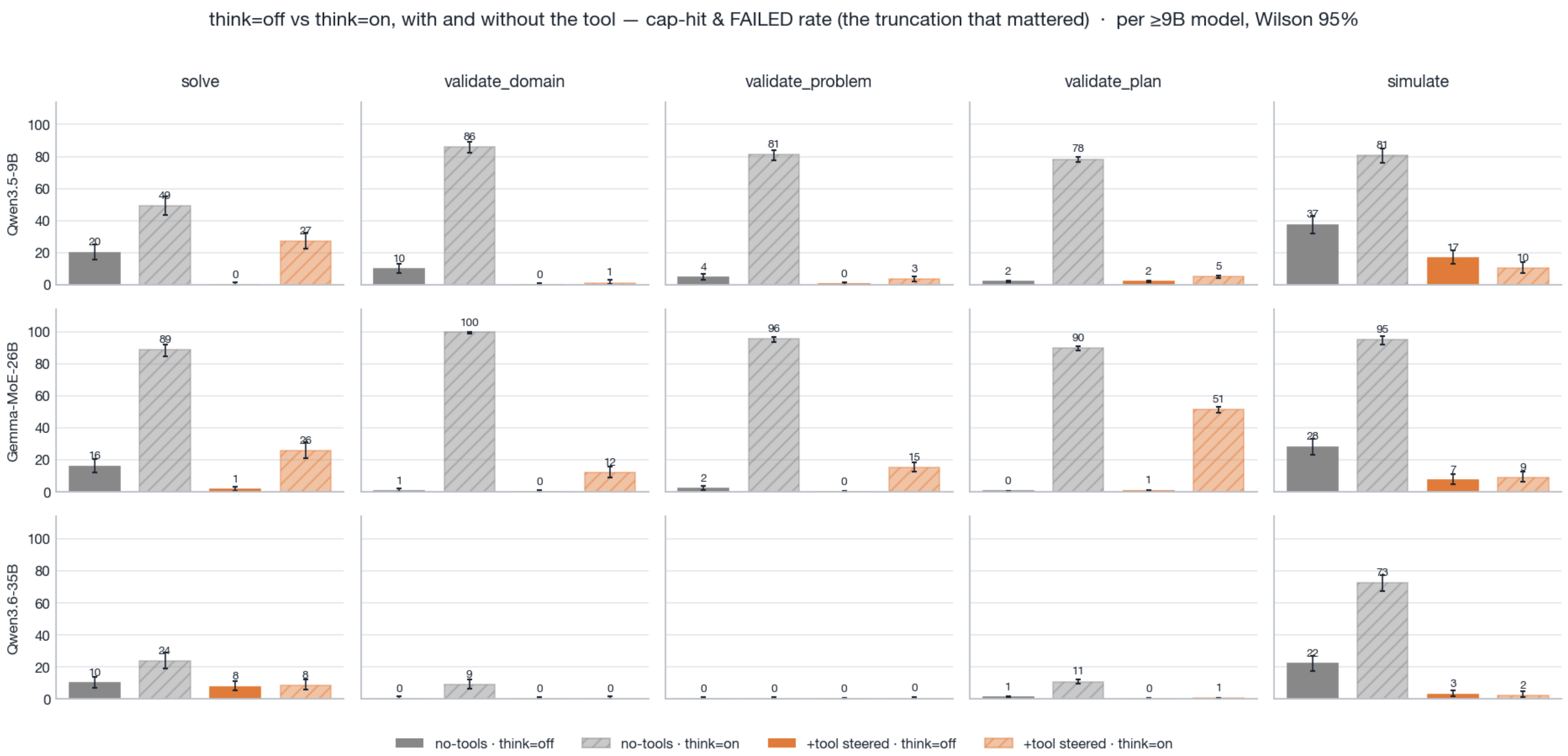
Left: solve tool-use rate — neutral think=on collapses tool-calling vs think=off (Gemma-MoE-26B 100→23, Qwen3.5-9B 100→59); steering partly restores it. Right: solve truncation spikes under think=on. Everything in this section sits under one shared 8,192-token decode budget: the unaided baseline often reasons past the cap and never answers, so raw think=on gaps are inflated by baseline truncation — the section therefore reads think=on only through budget-insensitive statistics (the robust floor, min over modes) and the budget-robust Qwen3.6-35B control.

The modes, side by side, per model — success



Rows = ≥9B models, columns = tasks; no-tools (grey) vs +tool steered (orange), solid = think=off, hatched = think=on, Wilson 95%. The steered tool arm barely moves across modes (35B: invariant on all five tasks; Gemma's residual validate_plan tax is the one exception), while the no-tools baseline collapses on the validation tasks under think=on — except 35B, the budget-robust control — and improves only on solve.

The modes, side by side, per model — the truncation that mattered



Same grid, same cells: share of trials that hit the 8,192-token cap AND failed. Under think=on the 9B/Gemma no-tools baselines lose 78–100% of validate_* trials and 81–95% of simulate trials to the cap, while 35B's validate_* baseline stays ≤11% — it survives the budget, its benefit barely moves across modes, and that proves the validate_* inflation is decode budget, not extra tool skill. The steered arm stays low everywhere except Gemma validate_plan (51%) — the same cell flagged as its residual budget tax.

Bottom line — one tool, three cross-mode regimes

Pairing every think=off cell with its think=on twin splits the five tasks into three regimes:

- solve — ROBUST. Realizable benefit ≥ +46pp in BOTH modes for every ≥9B model. The gap shrinks under think=on only because reasoning rescues the unaided baseline (9B 11→27%, 35B 9→38%), not because the tool weakens (35B steered: 92% in both modes).
- simulate — SOLE-SOURCE. The baseline is 0% in both modes; the tool delivers +83…+97pp. No budget choice can confound a benefit that has no baseline to move.
- validate_domain / problem / plan — BUDGET-DEPENDENT. The dramatic think=on gaps (Δ up to +67pp vs off) appear exactly where the no-tools baseline truncates 78–100% and collapses. The honest mode-invariant claim is the robust floor: validate_plan +5…+16pp, validate_problem +20…+25pp, validate_domain +21pp (Gemma) to +74pp (9B, whose off baseline is already weak).

→ The verdict pattern — RQ0.1–0.4 YES/YES/MIXED/YES, RQ0.5 YES, RQ0.6 NO — reproduces in both modes (asserted at build time). The MAGNITUDES do not: they move with the decode budget.

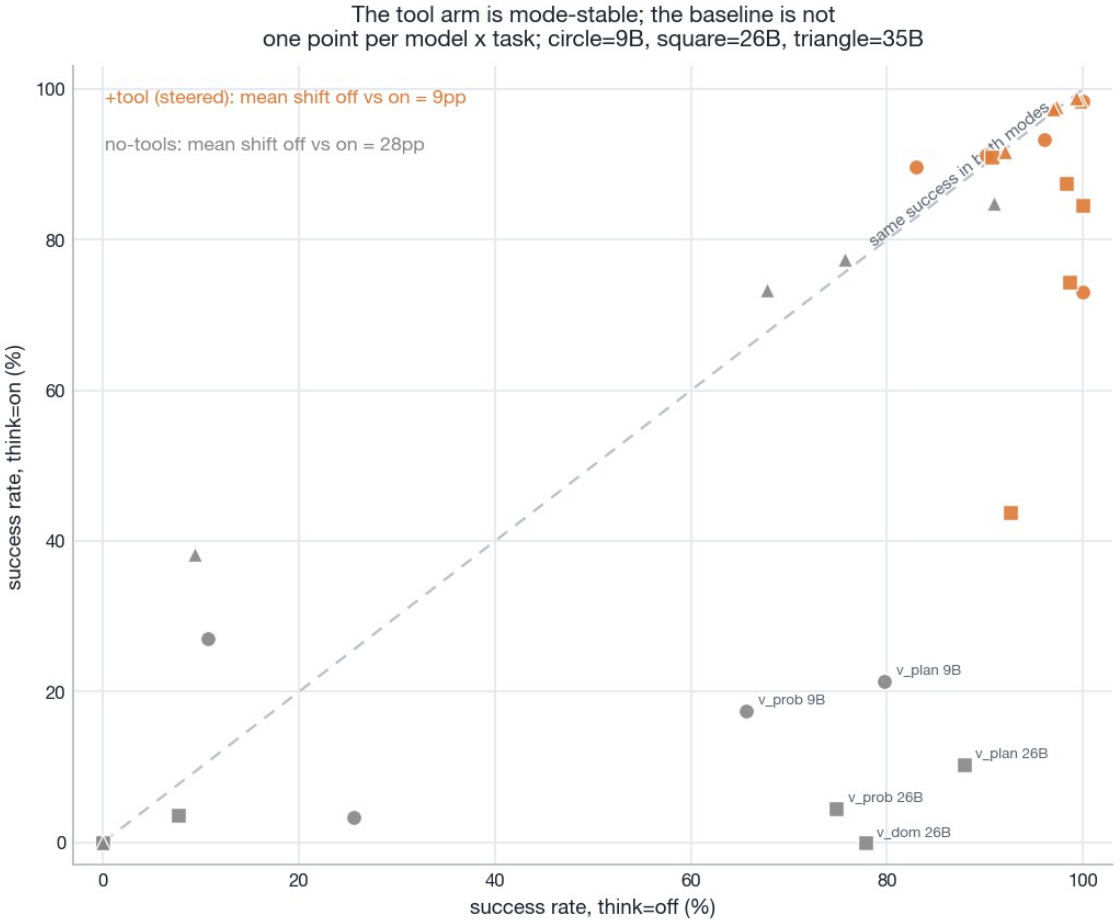
How this deck aggregates — and what it never does

Both source decks stay untouched: think=off is the locked headline, think=on the budget-confounded companion. This deck combines them only at the level of per-cell statistics.

- Spine metric: REALIZABLE benefit = success(+tool steered) − success(no-tools), per model × task × mode. Why the steered arm and not plain availability: two slides down.
- CI on each gap: Newcombe MOVER (Wilson-consistent difference of two independent proportions). CI on Δ(on−off): MOVER again on the two gaps — valid because the off and on corpora share no trials. '*' = 95% CI excludes 0.
- Robust floor = min over modes of the realizable benefit — the budget-insensitive lower bound on what the tool is worth. Classes: robust (every ≥9B floor ≥ +30pp) · sole-source (baseline floored in both modes) · budget-dep (some floor < +30pp).
- Raw trials are NEVER pooled across arms or think modes — each statistic comes from exactly one (model, mode, arm) corpus, and only the statistics are differenced or compared.

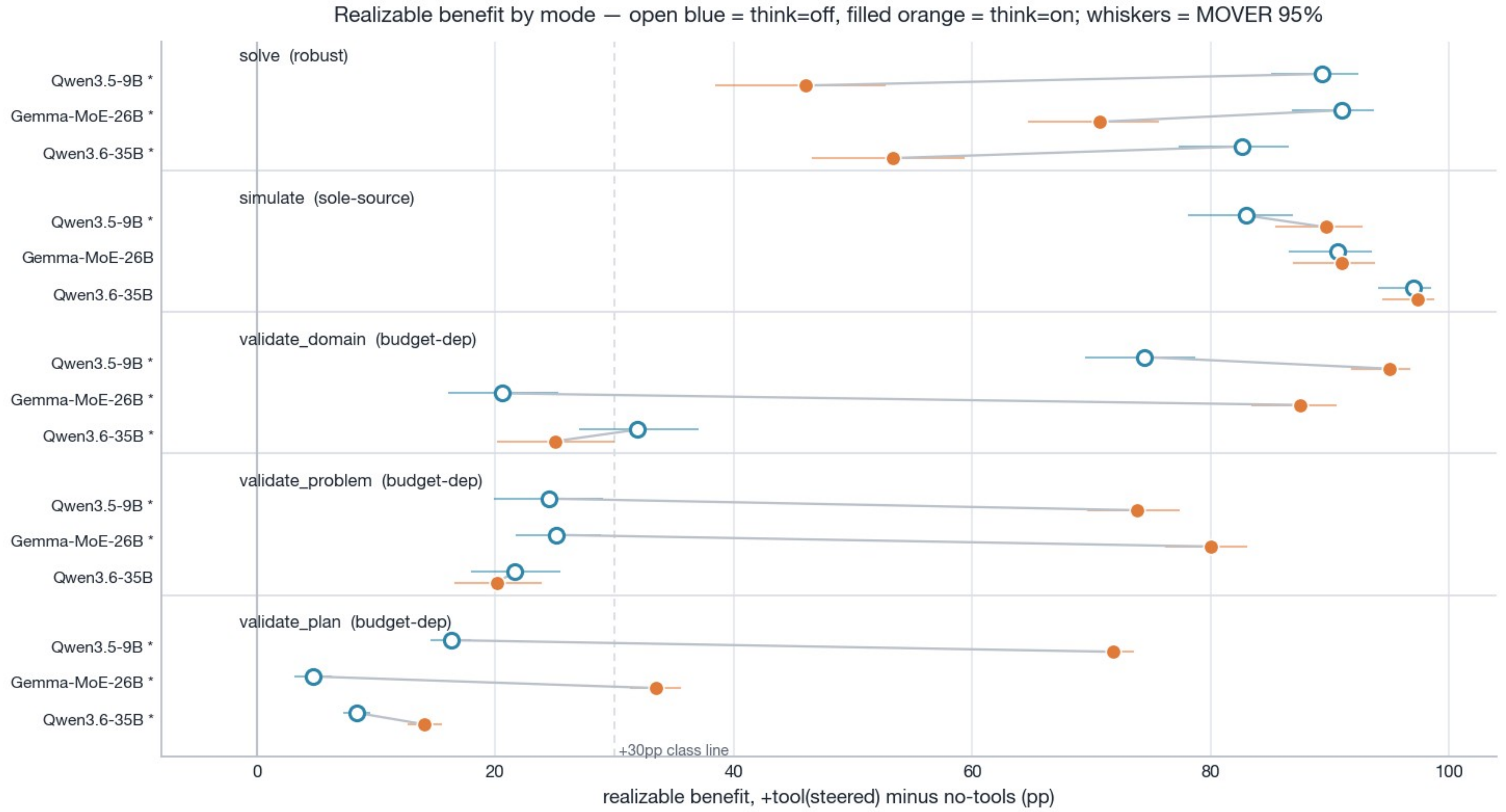
→ Standing caveats inherited from think=on: reasoning + answer share one 8,192-token decode budget (truncation differs by arm and mode — evidence slide near the end); completion tokens carry no logged thinking/answer split; latency is only defensible as turns × output tokens.

The gap moves because the BASELINE moves — not the tool arm



Success at think=off (x) vs think=on (y), one point per ≥9B model and task. The steered tool arm (orange) hugs the diagonal — mean shift 9pp, and Qwen3.6-35B's steered arm is fully mode-invariant on all five tasks. The no-tools baseline (grey) swings 28pp on average: down on the validation tasks (it reasons past the budget — Gemma validate_domain 78→0%, validate_plan 88→10%), up on solve (reasoning genuinely helps derivation). Every cross-mode gap change is driven by the grey points, so mode×arm interaction = baseline effect.

Realizable benefit by mode — what the floor protects



Benefit of the steered tool over no-tools, per ≥9B model: think=off (open blue) vs think=on (filled orange), MOVER 95% whiskers; '*' on the label = Δ(on−off) significant (MOVER-D 95% CI excludes 0). solve stays large in both modes (floor +46pp); simulate is the tool or nothing in both; the validate_* dumbbells stretch right under think=on — that stretch is the budget artifact, and the open-blue end (≈ the floor) is the defensible claim.

Cross-mode summary — five tasks, three classes (≥9B ranges)

task	realizable off (pp)	realizable on (pp)	robust floor (pp)	Δ(on−off), sig	class
solve	+83…+91	+46…+71	+46…+71	Δ -43…-20, 3/3	**robust**
simulate	+83…+97	+90…+97	+83…+97	Δ +0…+7, 1/3	**sole-source**
validate_domain	+21…+74	+25…+95	+21…+74	Δ -7…+67, 3/3	**budget-dep**
validate_problem	+22…+25	+20…+80	+20…+25	Δ -2…+55, 2/3	**budget-dep**
validate_plan	+5…+16	+14…+72	+5…+16	Δ +6…+56, 3/3	**budget-dep**

Why the spine is the STEERED arm, not plain availability

The per-mode decks lead with the availability gap (+tool plain vs no-tools, byte-identical wording). Across modes that gap stops measuring the tool:

- Under think=on the plain arm largely stops CALLING the tool — it reasons instead. solve tool_selected: Gemma 100→23%, 9B 100→59% (off→on); validate_plan: Gemma 21→1%.
- So a cross-mode availability comparison would conflate three things: the tool's value, the baseline's budget collapse, and a mode-dependent tool-calling failure that steering is known to repair (the off deck's RQ0.3 mechanism).
- The steered arm isolates 'what is the tool worth when actually invoked' — the quantity a deployment that prompts for tool use would realize. That is the only gap whose cross-mode difference is interpretable.
- Honesty note: steering does not fully de-confound Gemma — its steered validate_plan still only calls the tool 44% of the time under think=on (success 93→44% off→on), the one steered cell with a residual budget tax. 35B's steered arm is immune on all five tasks.

→ Availability and steering gaps remain fully reported, per mode, in the two source decks.

Cost-of-pass flips from a task regime to a model regime

task	model	off: no-tools	off: +tool steered	on: no-tools	on: +tool steered
solve	Qwen3.5-9B	46k	19k 0.4× cheaper	25k	26k 1.1× costlier
	Gemma-MoE-26B	50k	16k 0.3× cheaper	247k	22k 0.09× cheaper
	Qwen3.6-35B	50k	21k 0.4× cheaper	18k	19k 1.1× costlier
simulate	Qwen3.5-9B	— (0 succ)	24k only producer	— (0 succ)	20k only producer
	Gemma-MoE-26B	— (0 succ)	19k only producer	— (0 succ)	18k only producer
	Qwen3.6-35B	— (0 succ)	17k only producer	— (0 succ)	19k only producer
validate_domain	Qwen3.5-9B	9.3k	10k 1.1× costlier	200k	11k 0.06× cheaper
	Gemma-MoE-26B	1.9k	9.2k 4.9× costlier	— (0 succ)	12k only producer
	Qwen3.6-35B	3.1k	10k 3.2× costlier	6.0k	11k 1.8× costlier
validate_problem	Qwen3.5-9B	3.2k	15k 4.6× costlier	39k	15k 0.4× cheaper
	Gemma-MoE-26B	2.5k	11k 4.3× costlier	158k	14k 0.09× cheaper
	Qwen3.6-35B	2.8k	12k 4.4× costlier	4.1k	13k 3.3× costlier
validate_plan	Qwen3.5-9B	3.9k	14k 3.6× costlier	33k	16k 0.5× cheaper
	Gemma-MoE-26B	2.9k	13k 4.3× costlier	71k	27k 0.4× cheaper
	Qwen3.6-35B	3.7k	14k 3.8× costlier	6.0k	15k 2.5× costlier

Robustness — contamination probe (sweep-6)

- A separate anonymised-corpus sweep (sweep-6) re-ran the matrix on structurally-identical domains with every surface name renamed, to test memorisation.
- Headline (think=off): the clean no-tools-neutral probe is near-null — Δ(canonical − anon) success ≤1.3pp mean |Δ|, zero CI-disjoint task cells. No broad train-set contamination of the pure-model baseline where it has headroom — the availability gaps in this deck are not a contamination artifact.
- The probe's ONLY CI-disjoint cells (validate_plan, think=on) were a TOKENISATION artifact — anonymised prompts ran ~5% longer, so more trials truncated under the shared budget; success-given-completion was equal. Not memorisation — and a live demonstration of how budget exhaustion, not ability, moves think=on numbers (exactly the confound the cross-mode section controls for).

Limitations — what these results do and do not cover

- Single-tool-use scope: each trial poses ONE task with one relevant tool. Multi-tool chaining, agentic workflows, cross-benchmark generalisation (PlanBench) and published formalizer baselines (e.g. Huang & Zhang, ACL 2025) are deliberately out of scope (phase-3).
- Model roster: 5 open models from 2 families (Qwen, Gemma); headline claims rest on the three ≥9B models. No frontier/closed models.
- Decode budget: one fixed 8,192-token output cap. Every think=on number is budget-confounded; whether a larger budget closes the unaided think=on gap is UNTESTED (a cap-raised rerun is the open follow-up) — which is why cross-mode claims use the robust floor only.
- Steering = one fixed sentence appended to the request; broader prompt sensitivity is not swept in this corpus.
- Latency is not recoverable from a batched vLLM server (wall-clock confounded, per-token timings synthetic); turns × output tokens is the only proxy reported. A concurrency=1 TTFT/TPOT micro-benchmark is future work.
- Grading is strict and end-to-end (structured output, exact canonical-form equality): success conflates task ability with artifact/format adherence — deliberate (deployment-realistic), and the failure taxonomy separates the two where it matters (simulate decomposition, 0.8B mechanism).
- validate_plan with-tools scoring: the corpus postdates the 2026-05-25 FastMCP arg-error runtime fix; the defensive read-time relabel is verified inert on sweep5v2.

What the paper can claim — and how to say it

- Headline: report think=off as the clean, unconfounded read (locked verdicts YES/YES/MIXED/YES, RQ0.5 YES, RQ0.6 NO).
- Mode-invariance: the verdict PATTERN survives reasoning mode, and the robust floor is the quantified mode-invariant claim — solve ≥ +46pp, simulate +83…+97pp (sole source), validate_problem ≈ +20…+25pp, validate_plan ≈ +5…+16pp, validate_domain ≥ +21pp.
- Budget-dependence: the large think=on availability/realizable gaps on the validation tasks are decode-budget artifacts (baseline truncation 78–100% on 9B/Gemma), NOT extra tool capability — cite the 35B control. Never quote a think=on validate_* gap without this caveat.
- What moves with mode (report as findings, not contradictions): RQ0.5's headroom case (validate_plan +5→+27pp at off; solve +48→+65pp at on); cost-of-pass regime (task-determined at off, model-determined at on); steering importance (grows under think=on — plain tool-calling collapses).

→ Open question (untested): whether a larger decode budget closes the think=on baseline gap — needs a cap-raised rerun; until then the floor is the claim that needs no such experiment.

Backup — detailed tables

The slides that follow hold the full per-model numbers behind the token section:

- total tokens/trial (input+output) per task × model × arm, with cost multiples vs no-tools;
- cost-of-pass (total tokens per success) with bootstrap 95% CIs;
- the secondary completion-only generation-cost lens — output tokens over successes only. It excludes the re-fed tool input (the dominant tool cost) and failed-attempt tokens, so it flatters tools; it is a generation-length diagnostic, never the consumption headline;
- the RQ0.6 difficulty-bin table.
- the RQ0.1 success tables (their charts saturate near 100%);
- the RQ0.5 difficulty-bin table;
- the cross-mode per-model MOVER/MOVER-D detail and the truncation-by-cell table.

Backup — total tokens/trial (input+output) · ≥4B, think=off · 1/2

task	model	no-tools	+tool(plain)	+tool(steered)
solve	Qwen3.5-4B	5.7k (1.2k+4.5k)	25k (22k+2.7k) 4.3× costlier	25k (22k+2.8k) 4.4× costlier
	Qwen3.5-9B	4.9k (1.2k+3.7k)	21k (18k+2.4k) 4.2× costlier	19k (17k+2.3k) 3.9× costlier
	Gemma-MoE-26B	3.8k (1.2k+2.6k)	18k (16k+2.3k) 4.6× costlier	15k (13k+2.0k) 4.0× costlier
	Qwen3.6-35B	4.7k (1.2k+3.5k)	18k (14k+3.7k) 3.7× costlier	20k (17k+3.1k) 4.2× costlier
validate_domain	Qwen3.5-4B	922 (794+128)	10k (9.2k+885) 11.0× costlier	10k (9.2k+891) 11.0× costlier
	Qwen3.5-9B	2.4k (794+1.6k)	10.0k (9.2k+728) 4.2× costlier	10k (9.3k+729) 4.2× costlier
	Gemma-MoE-26B	1.4k (819+624)	9.0k (8.2k+799) 6.2× costlier	9.0k (8.2k+795) 6.2× costlier
	Qwen3.6-35B	2.1k (794+1.3k)	10k (9.5k+849) 4.9× costlier	10k (9.3k+748) 4.7× costlier
validate_problem	Qwen3.5-4B	1.3k (1.1k+114)	19k (16k+3.2k) 15.0× costlier	19k (15k+3.3k) 14.9× costlier
	Qwen3.5-9B	2.1k (1.1k+940)	13k (12k+1.3k) 6.3× costlier	13k (12k+1.4k) 6.3× costlier
	Gemma-MoE-26B	1.9k (1.2k+710)	11k (9.8k+1.2k) 5.8× costlier	11k (9.7k+1.2k) 5.7× costlier
	Qwen3.6-35B	2.1k (1.1k+962)	12k (11k+1.5k) 5.9× costlier	12k (11k+1.2k) 5.6× costlier

Backup — total tokens/trial (input+output) · ≥4B, think=off · 2/2

task	model	no-tools	+tool(plain)	+tool(steered)
validate_plan	Qwen3.5-4B	3.0k (1.4k+1.6k)	**31k (27k+4.1k) 10.3× costlier**	**26k (22k+3.6k) 8.7× costlier**
	Qwen3.5-9B	3.1k (1.4k+1.8k)	**14k (12k+1.5k) 4.5× costlier**	**14k (12k+1.4k) 4.4× costlier**
	Gemma-MoE-26B	2.5k (1.4k+1.1k)	**6.5k (5.3k+1.2k) 2.6× costlier**	**12k (10k+1.4k) 4.6× costlier**
	Qwen3.6-35B	3.4k (1.4k+2.0k)	**13k (11k+1.6k) 3.8× costlier**	**14k (12k+1.4k) 4.2× costlier**
simulate	Qwen3.5-4B	5.3k (1.4k+3.9k)	**26k (23k+3.3k) 4.9× costlier**	**30k (26k+3.7k) 5.6× costlier**
	Qwen3.5-9B	4.6k (1.4k+3.2k)	**18k (15k+2.2k) 3.8× costlier**	**20k (18k+2.0k) 4.3× costlier**
	Gemma-MoE-26B	4.8k (1.5k+3.3k)	**17k (15k+1.8k) 3.5× costlier**	**17k (16k+1.8k) 3.6× costlier**
	Qwen3.6-35B	4.0k (1.4k+2.6k)	**15k (13k+2.0k) 3.7× costlier**	**17k (15k+1.6k) 4.1× costlier**

Backup — cost-of-pass, tokens per success [bootstrap 95% CI] • ≥4B, think=off • 1/2

task	model	no-tools	+tool(plain)	+tool(steered)
solve	Qwen3.5-4B	74k [53k,120k]	32k [29k,36k] 0.4× cheaper	31k [29k,34k] 0.4× cheaper
	Qwen3.5-9B	46k [34k,68k]	21k [20k,22k] 0.5× cheaper	19k [18k,20k] 0.4× cheaper
	Gemma-MoE-26B	50k [35k,82k]	18k [17k,19k] 0.4× cheaper	16k [15k,16k] 0.3× cheaper
	Qwen3.6-35B	50k [37k,77k]	28k [26k,30k] 0.6× cheaper	21k [20k,23k] 0.4× cheaper
validate_domain	Qwen3.5-4B	4.8k [3.9k,6.2k]	11k [10k,11k] 2.2× costlier	11k [10k,11k] 2.2× costlier
	Qwen3.5-9B	9.3k [7.6k,12k]	10.0k [9.8k,10k] 1.1× costlier	10k [9.9k,10k] 1.1× costlier
	Gemma-MoE-26B	1.9k [1.7k,2.0k]	9.2k [9.0k,9.5k] 5.0× costlier	9.2k [9.0k,9.4k] 4.9× costlier
	Qwen3.6-35B	3.1k [2.9k,3.5k]	10k [10k,11k] 3.3× costlier	10k [9.9k,10k] 3.2× costlier
validate_problem	Qwen3.5-4B	2.2k [2.1k,2.4k]	24k [22k,26k] 10.7× costlier	23k [21k,25k] 10.3× costlier
	Qwen3.5-9B	3.2k [2.9k,3.5k]	14k [13k,14k] 4.3× costlier	15k [14k,15k] 4.6× costlier
	Gemma-MoE-26B	2.5k [2.4k,2.7k]	11k [11k,11k] 4.4× costlier	11k [11k,11k] 4.3× costlier
	Qwen3.6-35B	2.8k [2.6k,3.0k]	13k [12k,13k] 4.6× costlier	12k [12k,12k] 4.4× costlier

Backup — cost-of-pass, tokens per success [bootstrap 95% CI] · ≥4B, think=off · 2/2

task	model	no-tools	+tool(plain)	+tool(steered)
validate_plan	Qwen3.5-4B	4.1k [3.9k,4.2k]	**44k [43k,46k] 10.9× costlier**	**33k [32k,34k] 8.1× costlier**
	Qwen3.5-9B	3.9k [3.8k,4.0k]	**15k [15k,15k] 3.8× costlier**	**14k [14k,14k] 3.6× costlier**
	Gemma-MoE-26B	2.9k [2.8k,3.0k]	**32k [30k,34k] 11.0× costlier**	**13k [12k,13k] 4.3× costlier**
	Qwen3.6-35B	3.7k [3.6k,3.8k]	**16k [15k,16k] 4.2× costlier**	**14k [14k,14k] 3.8× costlier**
simulate	Qwen3.5-4B	— (0 succ)	43k [38k,50k]	51k [45k,60k]
	Qwen3.5-9B	— (0 succ)	27k [25k,30k]	24k [22k,26k]
	Gemma-MoE-26B	— (0 succ)	19k [17k,20k]	19k [18k,21k]
	Qwen3.6-35B	— (0 succ)	20k [19k,21k]	17k [16k,18k]

Backup — completion-only output tokens per success (secondary, tool-flattering) · 1/2

task	model	no-tools	+tool(plain)	+tool(steered)
solve	Qwen3.5-4B	2.8k [1.7k,3.9k]	2.4k [2.2k,2.6k]	2.5k [2.4k,2.7k]
	Qwen3.5-9B	1.9k [1.3k,2.6k]	2.4k [2.3k,2.6k]	2.3k [2.1k,2.4k]
	Gemma-MoE-26B	1.2k [575,2.1k]	2.3k [2.2k,2.4k]	2.0k [1.8k,2.1k]
	Qwen3.6-35B	2.2k [1.8k,2.7k]	3.2k [3.0k,3.4k]	2.9k [2.8k,3.1k]
validate_domain	Qwen3.5-4B	95 [45,155]	863 [772,961]	886 [785,996]
	Qwen3.5-9B	865 [694,1.1k]	728 [692,765]	729 [694,765]
	Gemma-MoE-26B	567 [544,594]	797 [760,836]	794 [756,832]
	Qwen3.6-35B	1.2k [1.1k,1.3k]	827 [767,892]	748 [711,786]
validate_problem	Qwen3.5-4B	108 [76,150]	2.5k [2.3k,2.8k]	2.7k [2.5k,3.0k]
	Qwen3.5-9B	619 [558,687]	1.3k [1.3k,1.4k]	1.4k [1.3k,1.4k]
	Gemma-MoE-26B	572 [555,589]	1.2k [1.2k,1.3k]	1.2k [1.2k,1.2k]
	Qwen3.6-35B	892 [840,947]	1.5k [1.4k,1.6k]	1.2k [1.1k,1.3k]

Backup — completion-only output tokens per success (secondary, tool-flattering) · 2/2

task	model	no-tools	+tool(plain)	+tool(steered)
validate_plan	Qwen3.5-4B	1.5k [1.5k,1.6k]	3.4k [3.3k,3.5k]	2.9k [2.9k,3.0k]
	Qwen3.5-9B	1.6k [1.6k,1.7k]	1.4k [1.4k,1.4k]	1.4k [1.3k,1.4k]
	Gemma-MoE-26B	1.1k [1.1k,1.1k]	1.0k [1.0k,1.1k]	1.4k [1.4k,1.4k]
	Qwen3.6-35B	1.9k [1.9k,2.0k]	1.6k [1.6k,1.6k]	1.4k [1.4k,1.5k]
simulate	Qwen3.5-4B	— (0 succ)	2.0k [1.8k,2.2k]	2.4k [2.1k,2.7k]
	Qwen3.5-9B	— (0 succ)	1.8k [1.7k,1.9k]	1.8k [1.6k,1.9k]
	Gemma-MoE-26B	— (0 succ)	1.5k [1.5k,1.6k]	1.6k [1.5k,1.7k]
	Qwen3.6-35B	— (0 succ)	1.7k [1.7k,1.8k]	1.5k [1.5k,1.6k]

Backup — RQ0.6 difficulty-binned success (object count)

task	bin	no-tools	+tool(steered)	Δ (pp)	n/arm
solve	≤6	14 [10,18]	100 [99,100]	+86	351
	6–14	8 [5,12]	97 [94,98]	+89	261
	>14	5 [3,8]	93 [90,96]	+89	288
validate_problem	≤5	73 [69,76]	94 [92,96]	+22	576
	5–13	73 [69,76]	96 [94,97]	+23	603
	>13	74 [71,78]	97 [95,98]	+23	540
validate_plan	≤6	83 [81,85]	96 [95,97]	+13	1755
	6–14	86 [84,87]	97 [96,98]	+12	1305
	>14	79 [77,81]	93 [92,94]	+14	1440
simulate	≤6	0 [0,1]	89 [86,92]	+89	351
	6–14	0 [0,1]	93 [89,95]	+93	261
	>14	0 [0,1]	89 [85,92]	+89	288

Backup — RQ0.1 validate_domain success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	19 [15,24]	94 [91,96]	96 [93,97]	**+75***	+1
Qwen3.5-9B	26 [21,30]	100 [99,100]	100 [99,100]	**+74***	+0
Gemma-MoE-26B	78 [73,82]	98 [95,99]	98 [96,99]	**+20***	+1
Qwen3.6-35B	68 [63,72]	99 [97,100]	100 [98,100]	**+31***	+1

Backup — RQ0.1 validate_problem success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	56 [53,60]	79 [76,82]	82 [79,85]	**+23***	+3
Qwen3.5-9B	66 [62,69]	95 [93,97]	90 [88,92]	**+30***	**-5***
Gemma-MoE-26B	75 [71,78]	100 [99,100]	100 [99,100]	**+25***	+0
Qwen3.6-35B	76 [72,79]	97 [95,98]	97 [96,98]	**+21***	+1

Backup — RQ0.4 simulate success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	0 [0,1]	60 [54,65]	58 [52,63]	**+60***	-2
Qwen3.5-9B	0 [0,1]	65 [59,70]	83 [78,87]	**+65***	**+18***
Gemma-MoE-26B	0 [0,1]	92 [88,94]	91 [87,93]	**+92***	-1
Qwen3.6-35B	0 [0,1]	75 [69,79]	97 [94,98]	**+75***	**+22***

Backup — RQ0.5 difficulty-binned success (plan length)

task	bin	no-tools	+tool(steered)	Δ (pp)	n/arm
solve	≤8	12 [9,16]	99 [97,100]	+87	324
	8–19	10 [7,14]	97 [94,98]	+87	279
	>19	6 [4,9]	95 [92,97]	+89	297
validate_plan	≤8	94 [93,95]	99 [99,100]	+5	1620
	8–19	91 [90,93]	98 [98,99]	+7	1395
	>19	62 [59,64]	89 [87,91]	+27	1485
simulate	≤8	0 [0,1]	100 [98,100]	+100	324
	8–19	0 [0,1]	93 [90,96]	+93	279
	>19	0 [0,1]	77 [72,82]	+77	297

Backup — per-model cross-mode detail — MOVER / MOVER-D 95% CIs · 1/2

task	model	realizable off	realizable on	Δ(on−off) [95%]	floor
solve	Qwen3.5-9B	+89 [+85,+92]*	+46 [+39,+53]*	Δ -43 [-51,-35]*	+46pp
	Gemma-MoE-26B	+91 [+87,+94]*	+71 [+65,+76]*	Δ -20 [-27,-14]*	+71pp
	Qwen3.6-35B	+83 [+77,+86]*	+53 [+47,+59]*	Δ -29 [-37,-21]*	+53pp
simulate	Qwen3.5-9B	+83 [+78,+87]*	+90 [+86,+93]*	Δ +7 [+1,+12]*	+83pp
	Gemma-MoE-26B	+91 [+87,+93]*	+91 [+87,+94]*	Δ +0 [-5,+5]	+91pp
	Qwen3.6-35B	+97 [+94,+98]*	+97 [+95,+99]*	Δ +0 [-3,+4]	+97pp
validate_domain	Qwen3.5-9B	+74 [+70,+79]*	+95 [+92,+97]*	Δ +21 [+15,+26]*	+74pp
	Gemma-MoE-26B	+21 [+16,+25]*	+88 [+84,+91]*	Δ +67 [+61,+72]*	+21pp
	Qwen3.6-35B	+32 [+27,+37]*	+25 [+20,+30]*	Δ -7 [-14,-0]*	+25pp

Backup — per-model cross-mode detail — MOVER / MOVER-D 95% CIs · 2/2

task	model	realizable off	realizable on	Δ(on−off) [95%]	floor
validate_problem	Qwen3.5-9B	**+24 [+20,+29]***	**+74 [+70,+77]***	Δ +49 [+43,+55]*	+24pp
	Gemma-MoE-26B	**+25 [+22,+29]***	**+80 [+76,+83]***	Δ +55 [+50,+59]*	+25pp
	Qwen3.6-35B	**+22 [+18,+25]***	**+20 [+17,+24]***	Δ -2 [-7,+4]	+20pp
validate_plan	Qwen3.5-9B	**+16 [+15,+18]***	**+72 [+70,+74]***	Δ +56 [+53,+58]*	+16pp
	Gemma-MoE-26B	**+5 [+3,+6]***	**+33 [+31,+36]***	Δ +29 [+26,+31]*	+5pp
	Qwen3.6-35B	**+8 [+7,+9]***	**+14 [+13,+15]***	Δ +6 [+4,+7]*	+8pp

Backup — the budget confound, cell by cell — truncation at the 8,192-token cap

task	model	no-tools off	no-tools on	steered off	steered on
solve	Qwen3.5-9B	20%	49%	35%	55%
	Gemma-MoE-26B	16%	92%	14%	35%
	Qwen3.6-35B	10%	24%	54%	31%
simulate	Qwen3.5-9B	37%	81%	74%	76%
	Gemma-MoE-26B	28%	95%	72%	75%
	Qwen3.6-35B	22%	73%	73%	78%
validate_domain	Qwen3.5-9B	10%	86%	0%	4%
	Gemma-MoE-26B	1%	100%	0%	12%
	Qwen3.6-35B	0%	9%	0%	1%
validate_problem	Qwen3.5-9B	5%	81%	1%	8%
	Gemma-MoE-26B	2%	96%	0%	18%
	Qwen3.6-35B	0%	0%	1%	4%
validate_plan	Qwen3.5-9B	2%	78%	10%	12%
	Gemma-MoE-26B	0%	90%	5%	52%
	Qwen3.6-35B	1%	11%	10%	10%